IF69D-S11 e OPEET014-OPEL - Processamento Digital de Imagens

Aula 10
Processamento de imagens coloridas

Universidade Tecnológica Federal do Paraná - UTFPR - Curitiba
Departamento Acadêmico de Eletrônica - DAELN
Curso de Engenharia Eletrônica
Prof. Humberto R. Gamba e Prof. Gustavo B. Borba

27.out.2026

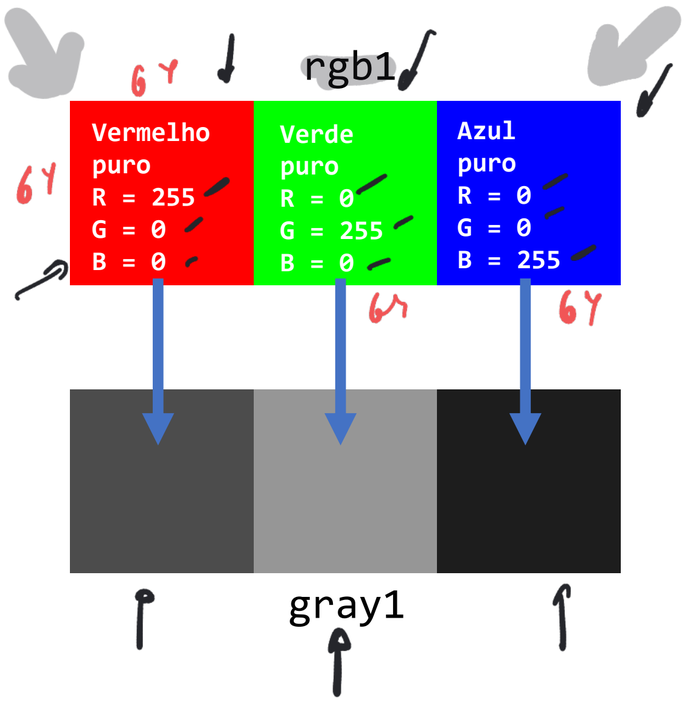
Imagens coloridas

$$\begin{pmatrix} Y \\ U \\ V \end{pmatrix} = \begin{pmatrix} 0.299 & 0.587 & 0.114 \\ -0.147 & -0.289 & 0.436 \\ 0.615 & -0.515 & -0.100 \end{pmatrix} \cdot \begin{pmatrix} R' \\ G' \\ B' \end{pmatrix}$$

$$\begin{pmatrix} Y' \\ C_b \\ C_r \end{pmatrix} = \begin{pmatrix} 0.299 & 0.587 & 0.114 \\ -0.169 & -0.331 & 0.500 \\ 0.500 & -0.419 & -0.081 \end{pmatrix} \cdot \begin{pmatrix} R' \\ G' \\ B' \end{pmatrix}$$

$$gray = 0.299 \cdot R' + 0.587 \cdot G' + 0.114 \cdot B'$$

Modelo	Aplicação Histórica	Componentes cromáticas
$Y'UV$	TV analógica PAL	U,V
YIQ	TV analógica NTSC	I, Q
$Y'CbCr$	Vídeo Digital, JPEG	Cb, Cr



Conversão de RGB para grayscale

```
1   % color_03 [script]
2   clear, clc, close all
3
4   m = ones(64).*255;
5   z = zeros(64);
6   r = cat(3,m,z,z); g = cat(3,z,m,z); b = cat(3,z,z,m);
7   rgb1 = uint8([r g b]); figure; imshow(rgb1);
8   gray1 = rgb2gray(rgb1); figure; imshow(gray1);
9
10  % Imagem: http://www.robots.ox.ac.uk/~vgg/data/
11  %         flowers/102/index.html
12  rgb2 = imread('image_03341.jpg'); figure, imshow(rgb2);
13  gray2 = rgb2gray(rgb2); figure; imshow(gray2);
```

rgb2

gray2

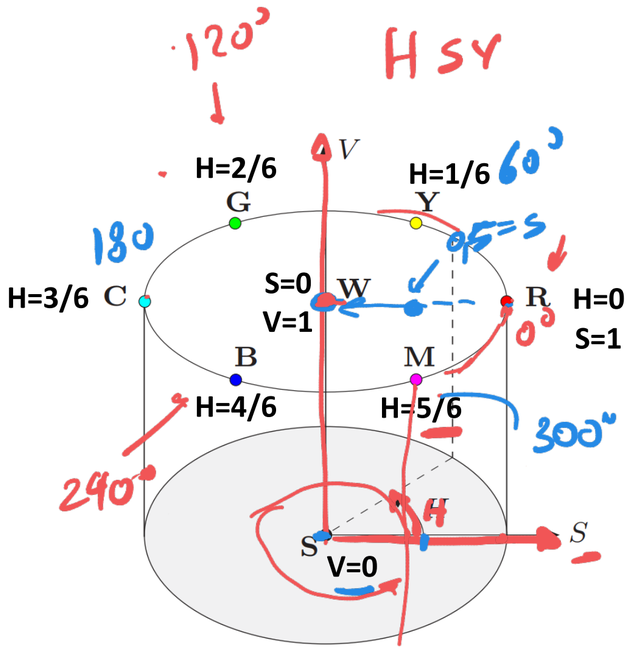
Imagens coloridas

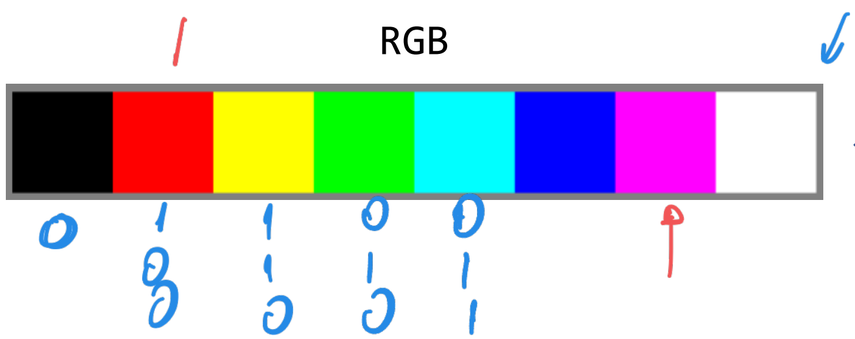
Conversão de RGB para HSV

Pt.	Color	R	G	B	H	S	V
S	Black	0.00	0.00	0.00	—	0.00	0.00
R	Red	1.00	0.00	0.00	0	1.00	1.00
Y	Yellow	1.00	1.00	0.00	1/6	1.00	1.00
G	Green	0.00	1.00	0.00	2/6	1.00	1.00
C	Cyan	0.00	1.00	1.00	3/6	1.00	1.00
B	Blue	0.00	0.00	1.00	4/6	1.00	1.00
M	Magenta	1.00	0.00	1.00	5/6	1.00	1.00
W	White	1.00	1.00	1.00	—	0.00	1.00

```
% color_04 [script]
clear, clc, close all
% Cores primárias e secundárias
%     S  R  Y  G  C  B  M  W
R = [0  1  1  0  0  0  1  1];
G = [0  0  1  1  1  0  0  1];
B = [0  0  0  0  1  1  1  1];

RGB = cat(3, R, G, B);
figure, image(RGB);
set(gcf,'color',[0.5 0.5 0.5]);
HSV = rgb2hsv(RGB);
```





	0	1/6	2/6	3/6	4/6	5/6	-
H	0	0.16	0.33	0.5	0.66	0.83	0
S	0	1	1	1	1	1	0
V	0	1	1	1	1	1	1

HSV(:,:,1) ➡ H
HSV(:,:,2) ➡ S
HSV(:,:,3) ➡ V

modelo HSV

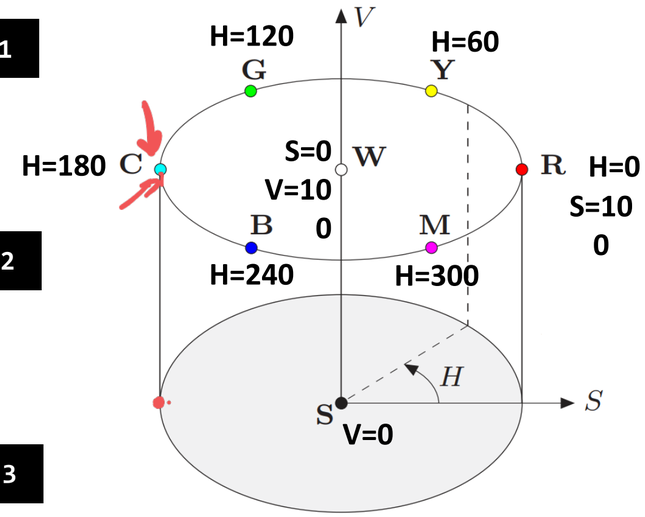
Figuras: Wilhelm Burger, Mark Burge, Digital image processing – an algorithmic introduction using Java, Springer, 2010, Fig 12.13.

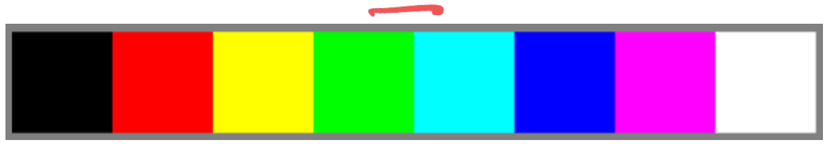
Imagens coloridas

Conversão de RGB para HSV

```
1    % color_04 [script]
2    clear, clc, close all
3    % Cores primárias e secundárias
4    %    S  R  Y  G  C  B  M  W
5    R = [0  1  1  0  0  0  1  1];
6    G = [0  0  1  1  1  0  0  1];
7    B = [0  0  0  0  1  1  1  1];
8
9    RGB = cat(3, R, G, B);
10   figure, image(RGB);
11   set(gcf,'color',[0.5 0.5 0.5]);
12   HSV = rgb2hsv(RGB);
13
14   % H [0...360], S e V [0...100]
15   Hc = HSV(:,:,1).*360;
16   Sc = HSV(:,:,2).*100;
17   Vc = HSV(:,:,3).*100;
```

http://colorizer.org



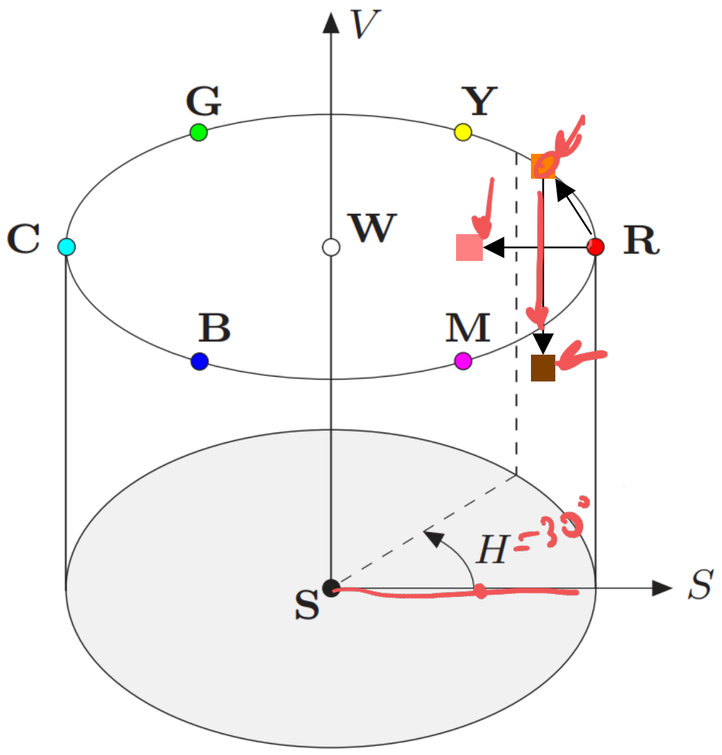


	Cor 1			Cor 2		Cor 3		
0	0	60	120	180	240	300	0	Hc ➡ H
0	100	100	100	100	100	100	0	Sc ➡ S
0	100	100	100	100	100	100	100	Vc ➡ V

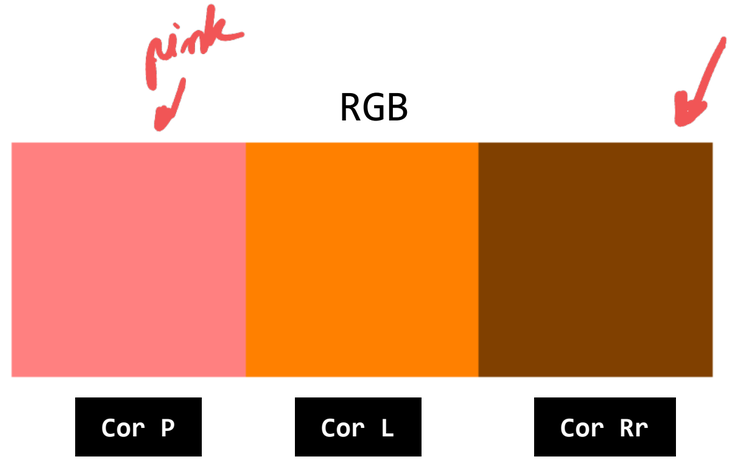
Figuras: Wilhelm Burger, Mark Burge, Digital image processing – an algorithmic introduction using Java, Springer, 2010, Fig 12.13.

Imagens coloridas

Conversão de RGB para HSV




```
1   % color_05 [script]
2   clear, clc, close all
3
4   % (P)Rosa: "vermelho esbranquiçado"
5   %           vermelho dessaturado
6   %                    low S
7   % (L)Laranja: entre o vermelho
8   %              e o amarelo
9   % (Rr)Marrom: "laranja escuro"
10  %                       low V
11  %      P      L      Rr
12  H = [ 0    1/12   1/12];
13  S = [0.5    1      1  ];
14  V = [ 1     1     0.5 ];
15
16  HSV = cat(3, H, S, V);
17  RGB = hsv2rgb(HSV);
18  figure, image(RGB);
```

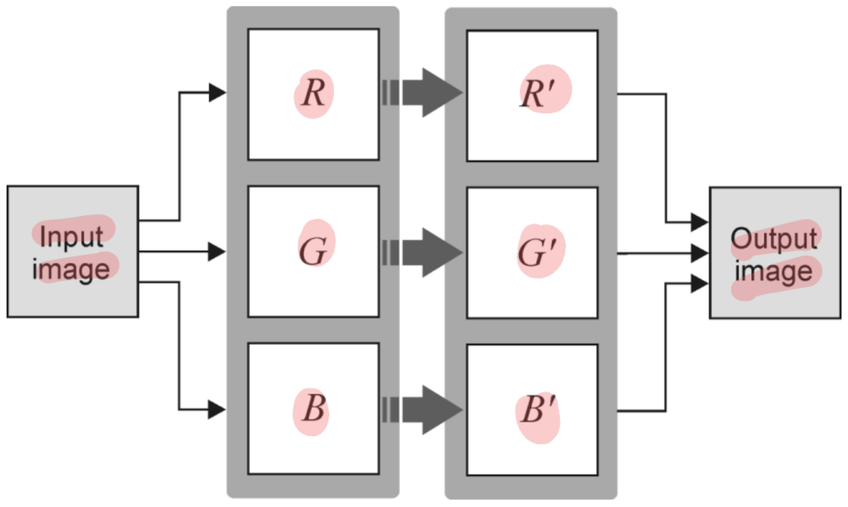


hsv2rgb( )

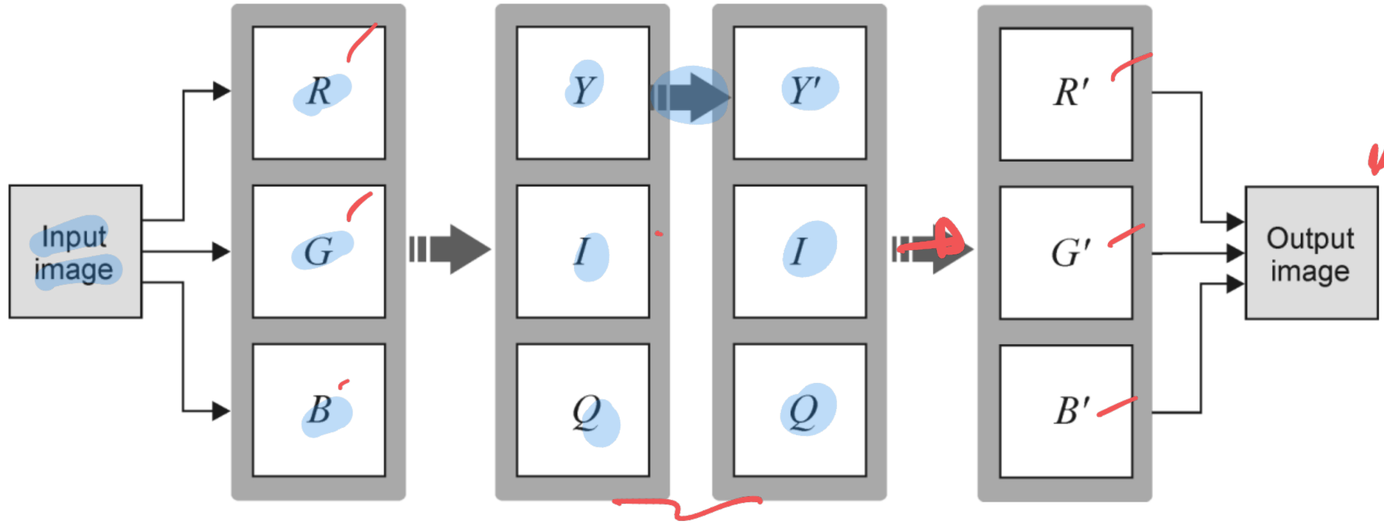
Figuras: Wilhelm Burger, Mark Burge, Digital image processing – an algorithmic introduction using Java, Springer, 2010, Fig 12.13.

Processamento de imagens coloridas

RGB processing



Intensity processing



preservamos
o I e Q
mantemos
a cromaticidade

Figuras: Oge Marques, Practical Image and Video Processing Using MATLAB, John Wiley and Sons, 2011, Fig 6.22 e Fig 16.23.

Processamento de imagens coloridas

RGB processing

- Suavização (filtro passa-baixas)

```
% color_06 [script]
clear, clc, close all
% Suavização (filtro passa baixas)
% Fazer a operação no canais R, G, B, separadamente
% Imagem:
% https://raw.githubusercontent.com/clausmichele/
% CBSD68-dataset/master/CBSD68/noisy25/0037.png
rgb = imread('0037.png');
figure, imshow(rgb), title('rgb')

r = rgb(:,:,1);
g = rgb(:,:,2);
b = rgb(:,:,3);
r_g_b = [r g b];
figure, imshow(r_g_b), title('r\_g\_b')

h = fspecial('gaussian', [5 5], 1);
rPB = imfilter(r, h);
gPB = imfilter(g, h);
bPB = imfilter(b, h);
r_g_b_PB = [rPB gPB bPB];
figure, imshow(r_g_b_PB), title('r\_g\_b\_PB')

rgbPB = cat(3, rPB, gPB, bPB);
figure, imshow(rgbPB), title('rgbPB')
```

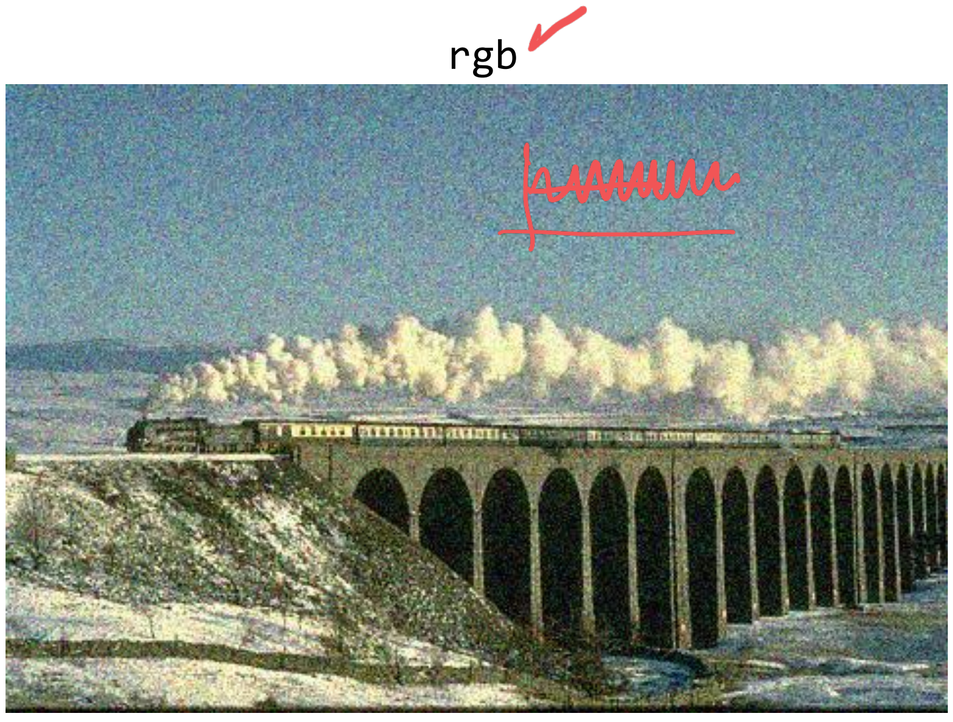


Imagem: https://raw.githubusercontent.com/clausmichele/CBSD68-dataset/master/CBSD68/noisy25/0037.png

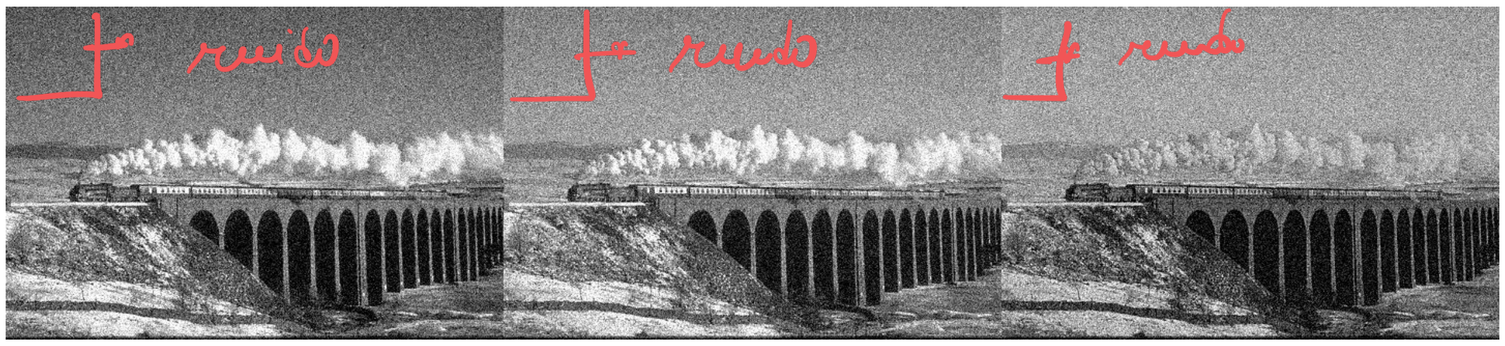
Processamento de imagens coloridas

RGB processing

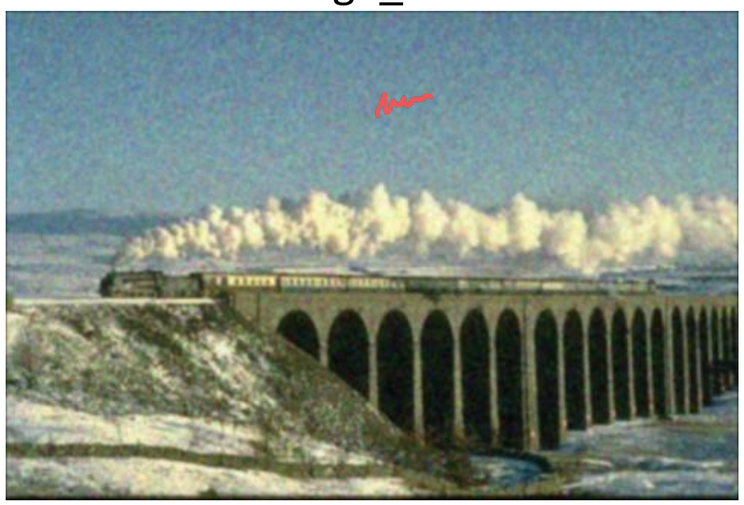
- Suavização (filtro passa-baixas)

rgb

r_g_b



rgb_PB



r_g_b_PB

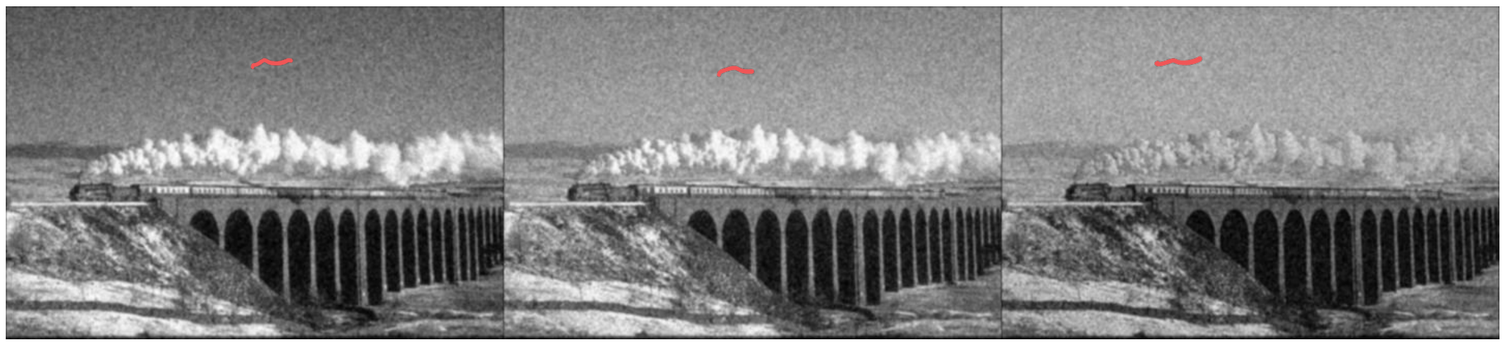


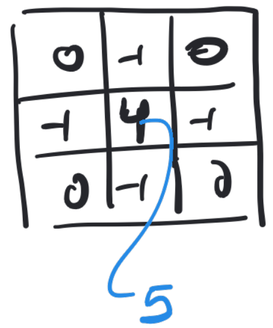
Imagem: https://raw.githubusercontent.com/clausmichele/CBSD68-dataset/master/CBSD68/noisy25/0037.png

Processamento de imagens coloridas

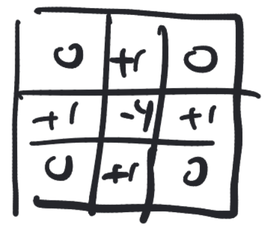
RGB processing

- Realce (sharpening)

```
1    % color_07 [script]
2    clear, clc, close all
3    % Realce (sharpening, filtro passa-altas)
4    % Fazer a operação nos canais R,G,B, separadamente
5    % Imagem: https://www2.eecs.berkeley.edu/Research/
6    % Projects/CS/vision/bsds/BSDS300/html/dataset/
7    % images/color/241004.html
8    rgb = imread('241004.jpg');
9    figure, imshow(rgb), title('rgb')
10   r = rgb(:,:,1);
11   g = rgb(:,:,2);
12   b = rgb(:,:,3);
13   r_g_b = [r g b];
14   figure, imshow(r_g_b), title('r\_g\_b')
15
16   h = -fspecial('laplacian',0);
17   rPA = imfilter(double(r), h);
18   gPA = imfilter(double(g), h);
19   bPA = imfilter(double(b), h);
20   rS = uint8(double(r) + double(rPA));
21   gS = uint8(double(g) + double(gPA));
22   bS = uint8(double(b) + double(bPA));
23   r_g_b_S = [rS gS bS];
24   figure, imshow(r_g_b_S), title('r\_g\_b\_S')
25
26   rgb_S = cat(3, rS, gS, bS);
27   figure, imshow(rgb_S), title('rgb\_S')
```



∇I



rgb

Imagem: Berkeley BSDS300 dataset https://www2.eecs.berkeley.edu/Research/Projects/CS/vision/bsds/BSDS300/html/dataset/images/color/241004.html

Processamento de imagens coloridas

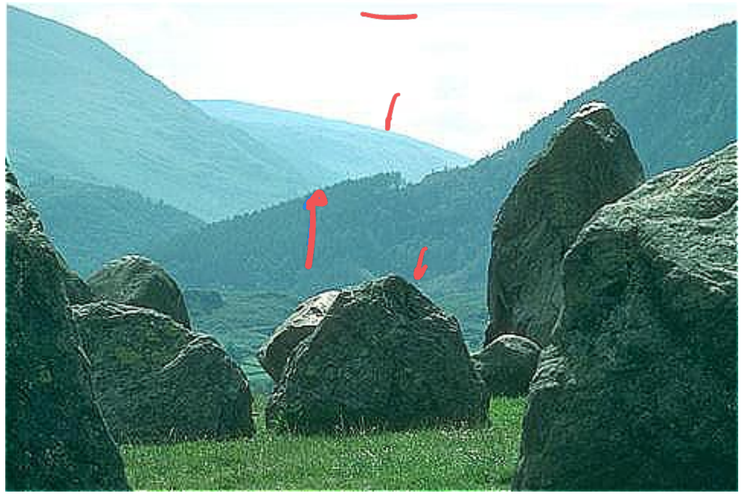
RGB processing

- Realce (sharpening)

rgb

r_g_b

rgb_S



r_g_b_S

Imagem: Berkeley BSDS300 dataset https://www2.eecs.berkeley.edu/Research/Projects/CS/vision/bsds/BSDS300/html/dataset/images/color/241004.html

Como melhorar o contraste de uma imagem colorida sem modificar significativamente suas cores?

Espaço	Componente equalizada	Componentes preservadas
RGB	R, G e B separadamente	Nenhuma
HSV	V (Value)	H, S
YIQ	Y' (luma)	I, Q
YCbCr	Y' (luma)	Cb, Cr
HSI (Gonzalez)	I (Intensidade)	H, S

Fundamentos:

1) A componente V do HSV não é a intensidade I do HSI.

$$V = máx\ (R, G, B)$$

$$I_{HSI} = \frac{R + G + B}{3}$$

2) No YIQ, para sinais R'G'B' normalizados:

$$Y' = 0.299\ R' + 0.587\ G' + 0.144\ B'$$

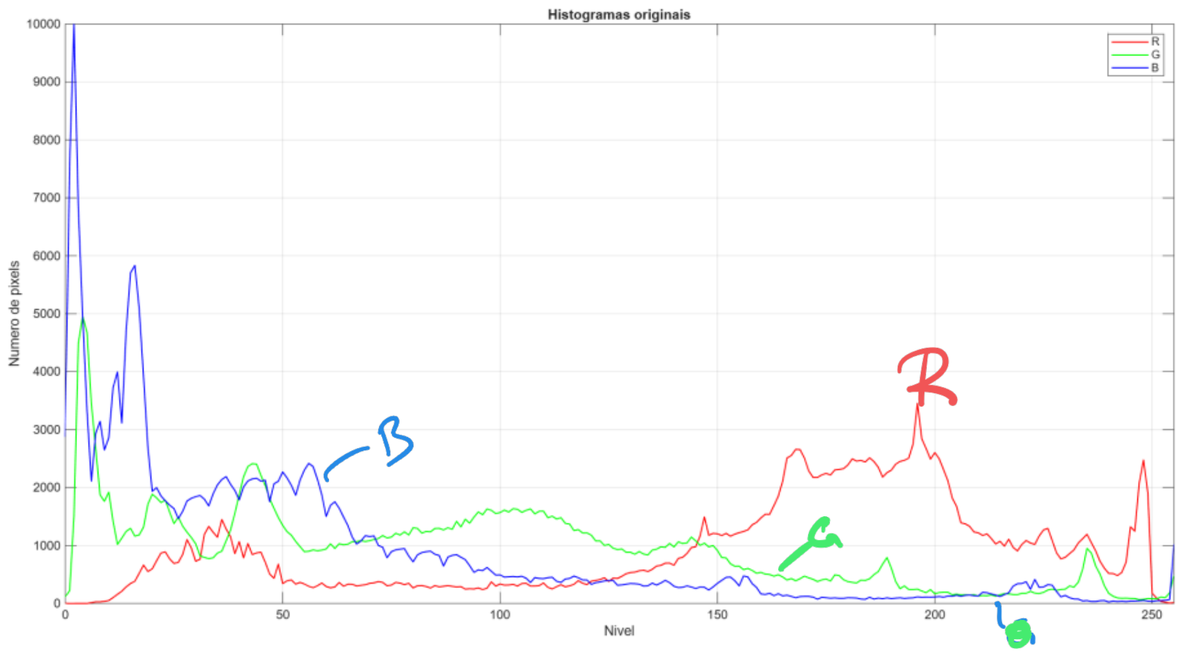
3) Preservar as componentes cromáticas durante a equalização não garante que a imagem RGB reconstruída preserve exatamente as cores.

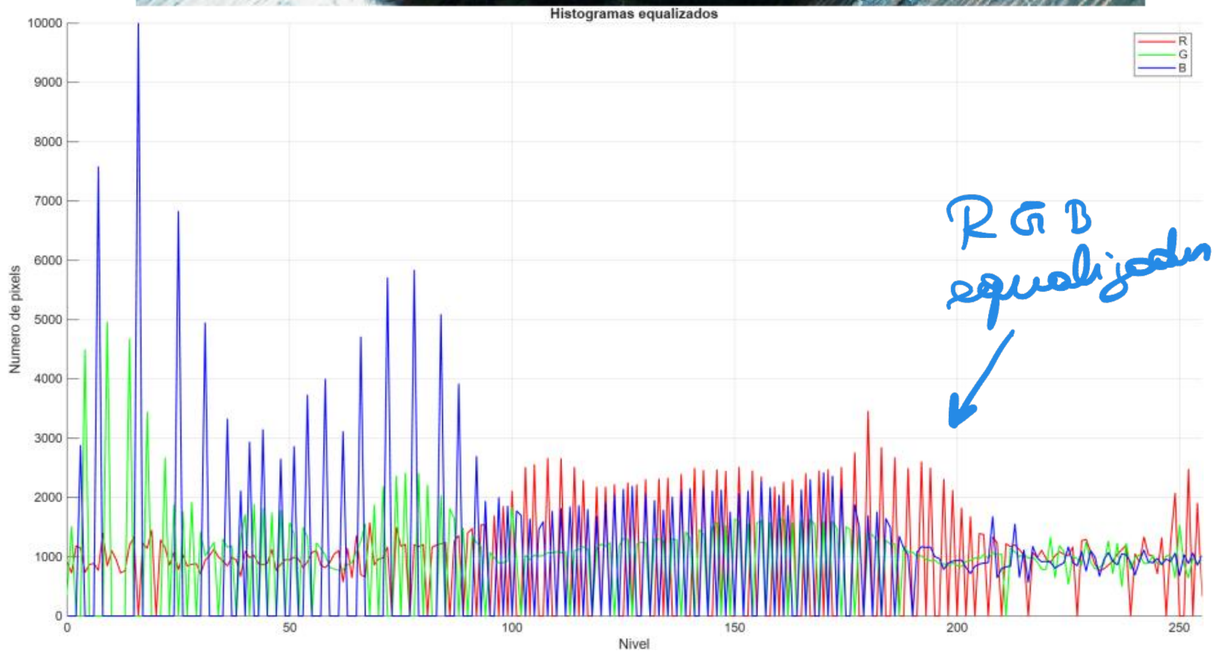
Isso acontece porque algumas transformações podem produzir valores RGB fora do intervalo permitido. A limitação desses valores (clipping) pode modificar as cores finais.

Equalização independente dos canais RGB

Cada componente é transformada pela sua própria distribuição acumulada (CDF).

Mudanca média de matiz (graus): 83.95°





Equalização independente dos canais RGB

Quantização da modificação cromática no espaço HSV:

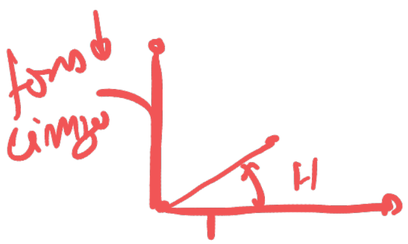
HSV0 = rgb2hsv(RGB); → imagem original
HSV1 = rgb2hsv(RGB_eq); → imagem equalizada

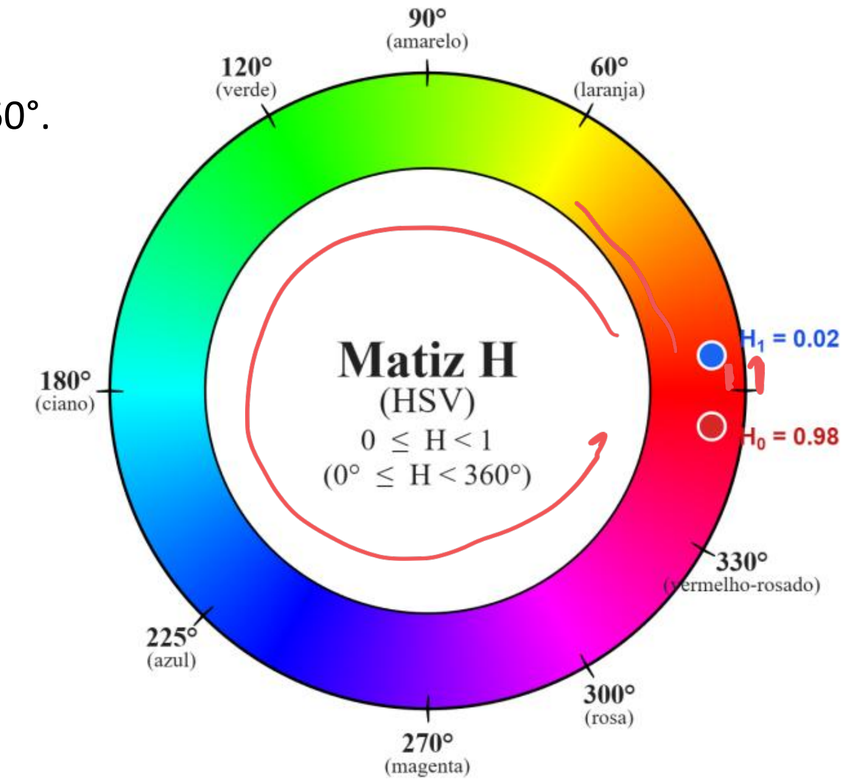
HSV0(:,:,1) → componente H da imagem original
HSV1(:,:,1) → componente H da imagem equalizada

$$dH(x,y) = |H_1(x,y) - H_0(x,y)|$$

No MATLAB, a componente H varia entre 0 e 1, representando um ângulo de 0° a 360°.
Exemplo:

Matiz original: 0,98 (352,8°).
Matiz equalizada: 0,02 (7,2°).
Diferença calculada: 0,96 (345,6°).
Diferença angular real: apenas 0,04 (14,4°).





HSV0 = **rgb2hsv**(RGB);
HSV1 = **rgb2hsv**(RGB_eq);
dH = abs(HSV1(:,:,1)-HSV0(:,:,1)); → **matriz com a diferença absoluta de matiz (Hue) entre duas imagens**
dH = min(dH,1-dH);
Locais_S_15 = HSV0(:,:,2)>0.15 & HSV1(:,:,2)>0.15; → **Seleciona saturação > 15%, menores ficam perto do cinza e a matiz fica pouco definida**
fprintf('Mudanca media absoluta de matiz (graus): %.2f\n', 360*mean(dH(Locais_S_15)));

Equalização do histograma no espaço HSV

No espaço HSV, a equalização é aplicada somente à componente V (Value), preservando as componentes H (matiz) e S (saturação).

V não é a luminância, mas o maior valor entre os componentes R, G e B: $V = \max(R, G, B)$

Imagem RGB orginal

↓

Conversão $RGB \rightarrow HSV \rightarrow (H, S, V)$

↓

H	S	$V_{eq} = T(V)$
Preservada	Preservada	**histeq(V)**

↓

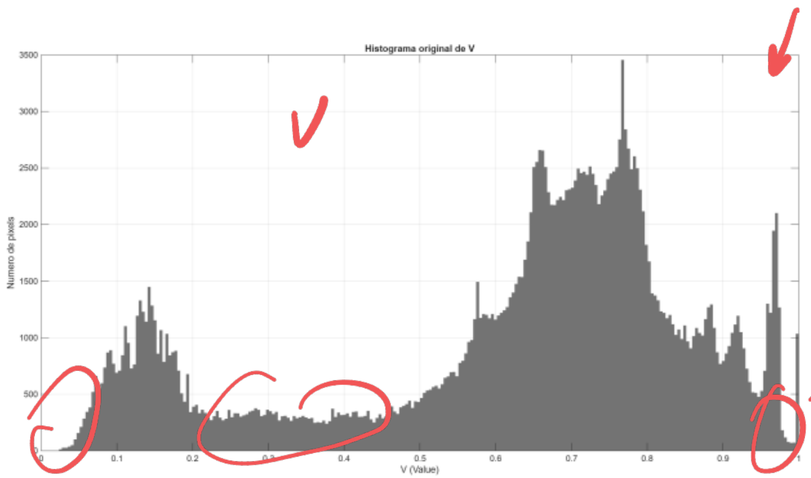
$(H, S, V_{eq}) \rightarrow (RGB)_{eq}$

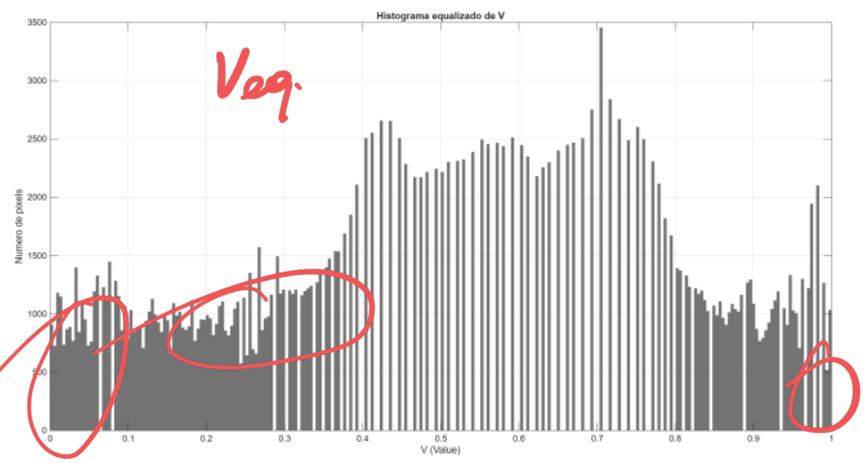
Imagem RGB equalizada

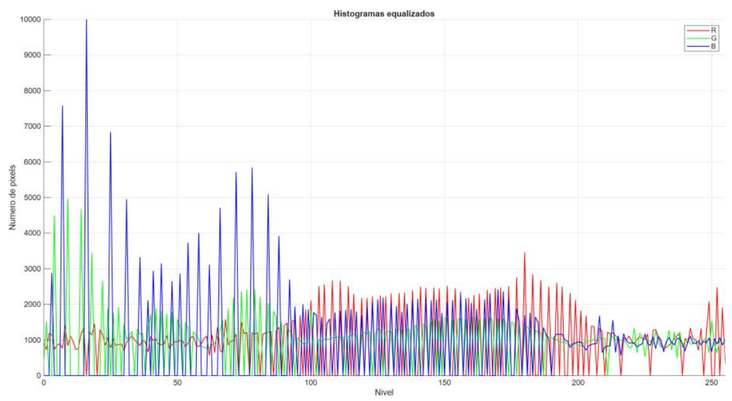
Equalização do histograma no espaço HSV

Equalização da componente V no espaço HSV

Mudança média de matiz: $H \approx 0°$







A equalização de V modifica o brilho e o contraste da imagem, alterando a aparência visual das cores, mas preservando sua matiz H e saturação S no modelo HSV.

Equalizacao_HSV_Cafe_Diferenca_Matiz.m

Equalização do histograma no espaço YIQ

A equalização é aplicada somente à luma Y’, preservando as componentes cromáticas I e Q.

Conversão $RGB \rightarrow YIQ$

Para componentes R', G' e B' normalizadas:

$$Y' = 0.299R' + 0.587\,G' + 0.114\,B'$$
$$I = 0.596\,R' - 0.274\,G' - 0.322\,B'$$
$$Q = 0.211\,R' - 0.523\,G' + 0.312\,B'$$

Y': luma

I, Q: cromaticidade

Imagem RGB original

Conversão $RGB \rightarrow YIQ$

$Y'_{eq} = histeq(Y')$

I e Q ficam inalterados

Conversão $YIQ \rightarrow RGB$

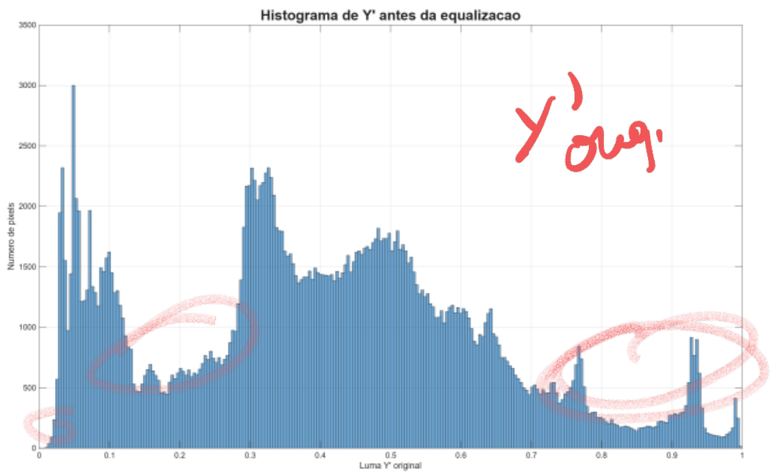
Para imagens RGB convencionais codificadas em sRGB (standard RGB), os valores armazenados já são não lineares (R’G’B’) e podem ser utilizados diretamente na conversão YIQ.

sRGB é um espaço de cores RGB padronizado, utilizado em imagens digitais. Seus componentes são codificados de maneira não linear e, por isso, podem ser representados por R’, G’ e B’.

RGB = imread('peppers.png'); → armazena R’, G’ e B’

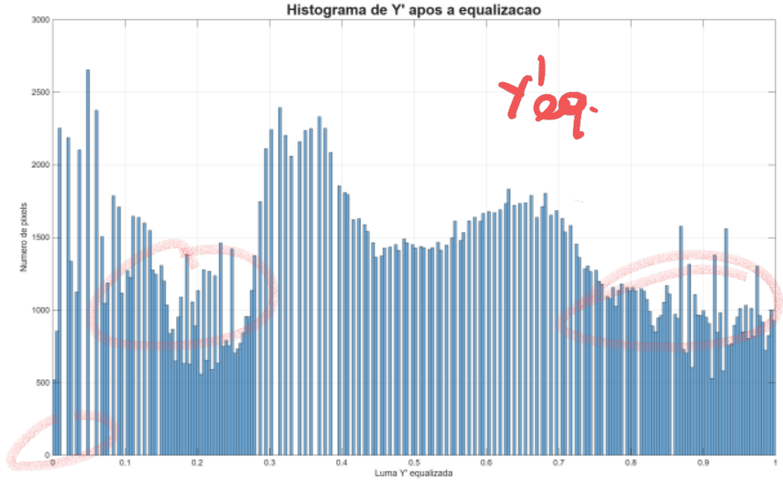
RGB linear representa componentes proporcionais à luz
R’G’B’ representa componentes codificadas não linearmente

Equalização do histograma no espaço YIQ



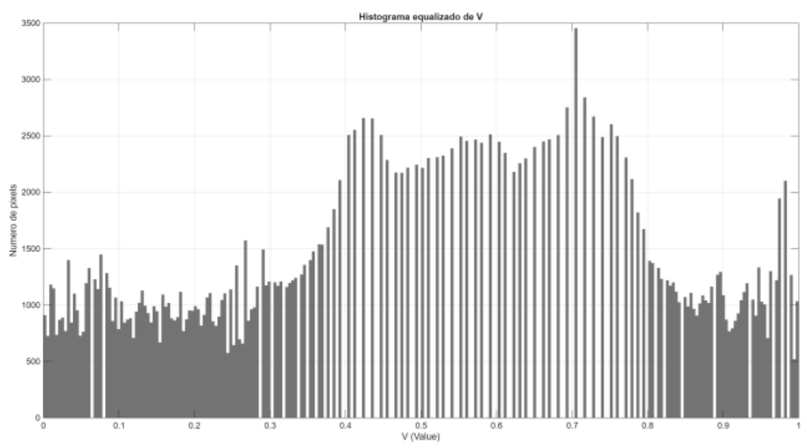
predominam tons quentes, com regiões de sombra

Mudança média de matiz: $H \approx 0°$



houve aumento da luminosidade: na madeira e nas partes claras da xícara

Equalização do histograma no espaço HSV



a imagem apresenta sombras mais intensas

Equalizacao_YIQ_Cafe.m

```
YIQ0 = rgb2ntsc(RGB0);
Y0 = YIQ0(:,:,1); % Luma Y'
Yeq = histeq(Y0, 256);
```

```
YIQeq = YIQ0;
YIQeq(:,:,1) = Yeq;
```

Equalização do histograma no espaço $Y'CbCr$

A equalização é aplicada somente à luma Y', preservando as componentes de diferença de cor Cb e Cr.

Conversão RGB → Y'CbCr (BT.601)

Para componentes R', G' e B' normalizadas:

$$Y' = 0{,}299\,R' + 0{,}587\,G' + 0{,}114\,B'$$

$$Cb* = \frac{(B' - Y')}{1.772}$$

$$Cr* = \frac{(R' - Y')}{1.402}$$

Y': luma | Cb*, Cr*: diferenças de cor
*forma normalizada, antes da codificação digital

Imagem RGB original

Conversão $RGB \rightarrow Y'CbCr$

$Y'_{eq} = histeq(Y')$

Cb e Cr ficam inalterados

Conversão $Y'CbCr \rightarrow RGB$

Imagem RGB equalizada

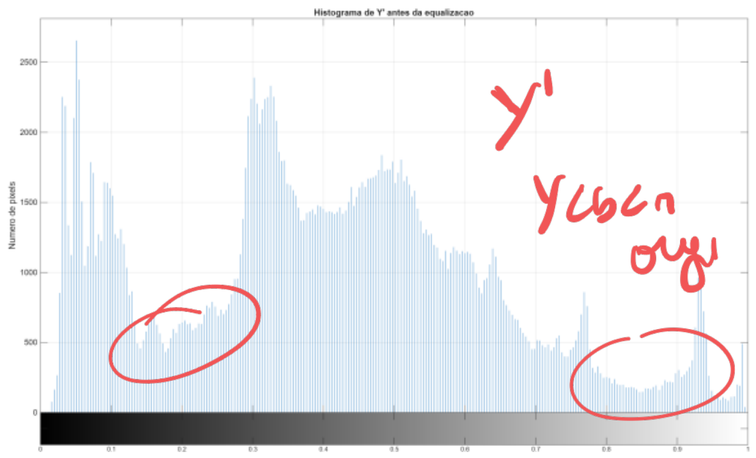
```
YCbCr0 = rgb2ycbcr(RGB0);
Y0 = YCbCr0(:,:,1);
Yeq_norm = histeq(Y0,256);
```

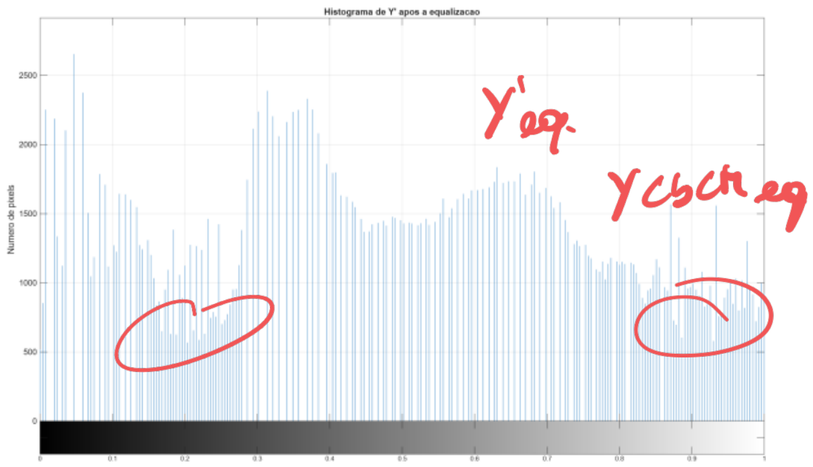
```
YCbCreq = YCbCr0;
YCbCreq(:,:,1) = Yeq;
RGB_eq = ycbcr2rgb(YCbCreq);
```

No MATLAB, rgb2ycbcr utiliza a codificação BT.601.

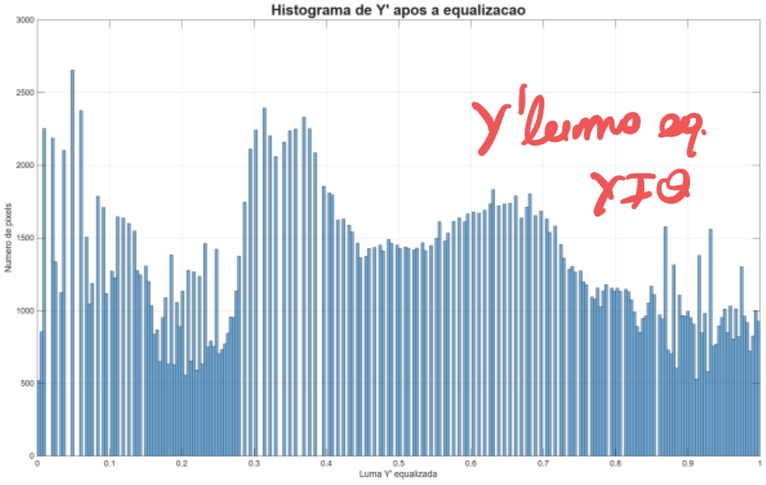
Equalização do histograma no espaço $Y'CbCr$

Mudança média de matiz: ≈ 42°





Y'IQ



Os modelos YIQ e Y'CbCr utilizam a mesma definição de luma Y', considerando BT.601.

Por isso, a equalização dessa componente produz resultados semelhantes, apesar de os modelos utilizarem representações diferentes da crominância.

Equalização do histograma no espaço HSI (Gonzales e Woods)

A equalização é aplicada à intensidade I, preservando inicialmente a matiz H e a saturação S.

Conversão RGB → HSI

Para componentes R, G e B normalizadas:

$$I = \frac{R + G + B}{3}$$

$$S = 1 - \frac{3 \cdot \min(R, G, B)}{R + G + B}$$

H é obtida das relações angulares entre R, G e B.

I: intensidade | H: matiz | S: saturação

Gonzalez, R. C.; Woods, R. E. Digital Image Processing. 3rd ed. Pearson, 2008.
Capítulo 6 — Color Image Processing, seção 6.5.5 — Histogram Processing.

Imagem RGB original

Conversão $RGB \rightarrow HSI$

$I_{eq} = histeq(I)$

H e S ficam inalterados na equalização

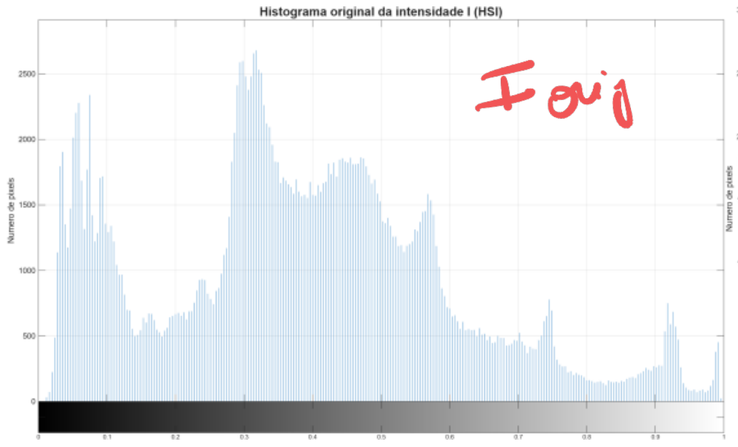
Conversão $HSI \rightarrow RGB$

Imagem RGB equalizada

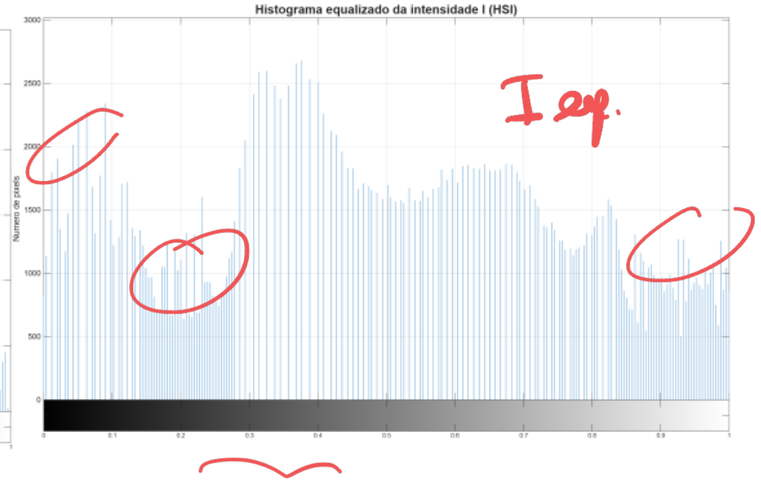
Diferentemente da luma Y' (YIQ / $Y'CbCr$), a intensidade I atribui o mesmo peso aos três canais RGB.

A expressão de S é válida quando $(R + G + B) > 0$. Para o preto, convenciona-se $S = 0$.

Equalização do histograma no espaço HSI

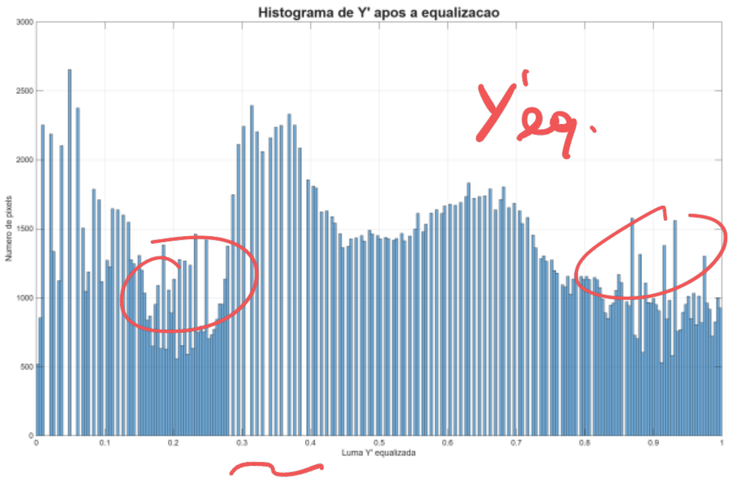


apresenta tons quentes e regiões de sombra



apresenta aumento do brilho em várias regiões, como na madeira e no interior da xícara.

Comparação com a equalização de Y’ no espaço YIQ



apresenta resultado semelhante, porém com algumas regiões com diferenças perceptíveis.

Processamento de imagens coloridas

Intensity processing

- Equalização do histograma

```
% color_08 [script]
clear, clc, close all
% Equalização do histograma
% Converter de RGB para um espaço de cores que separe a informação
% de luminância da informação de cor (cromaticidade),
% realizar a operação apenas no canal de luminância e
% converter para RGB.
% Espaço de cores YIQ, Y é a luminância
% |Y| |0,299  0,587  0,114| |R|
% |I|=|0,596 -0.274 -0,322|.|G|
% |Q| |0.211 -0,523  0,312| |B|
% No MATLAB: YIQ=rgb2ntsc(RGB), RGB=ntsc2rgb(YIQ)
% Imagem: https://www2.eecs.berkeley.edu/Research/
% Projects/CS/vision/bsds/BSDS300/html/dataset/
% images/color/210088.html
rgb = imread('210088.jpg');
figure, imshow(rgb), title('rgb')

yiq = rgb2ntsc(rgb);
y = yiq(:,:,1);
figure, imshow(y), title('y')
y_Eq = histeq(y,256);
figure, imshow(y_Eq), title('y\_Eq')

yiq_Eq = cat(3,y_Eq,yiq(:,:,2),yiq(:,:,3));
rgb_Eq = ntsc2rgb(yiq_Eq);
figure, imshow(rgb_Eq); title('rgb\_Eq')
```

rgb

Imagem: Berkeley BSDS300 dataset https://www2.eecs.berkeley.edu/Research/Projects/CS/vision/bsds/BSDS300/html/dataset/images/color/210088.html

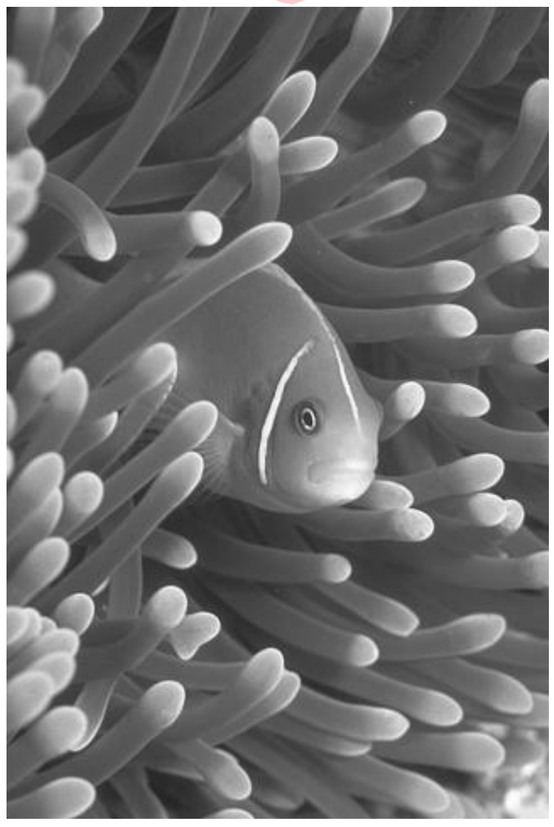
Processamento de imagens coloridas

Intensity processing

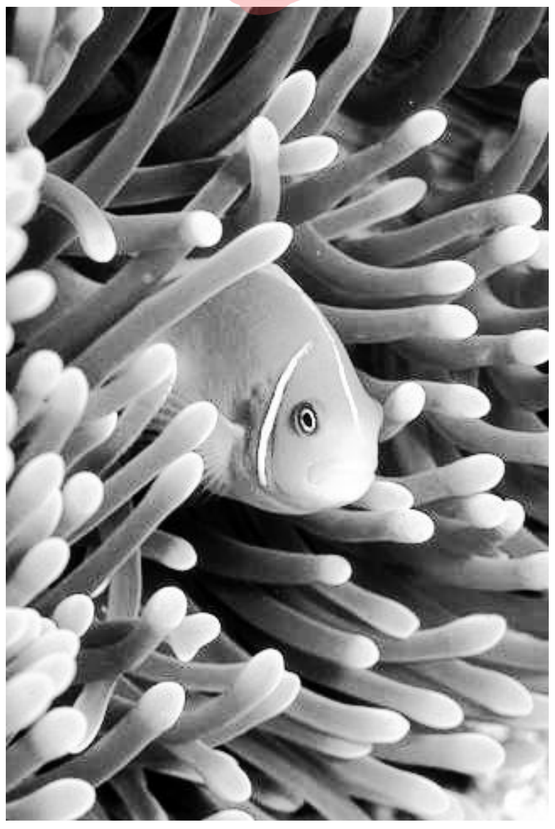
- Equalização do histograma

rgb

y



y_Eq



rgb_Eq

Imagem: Berkeley BSDS300 dataset https://www2.eecs.berkeley.edu/Research/Projects/CS/vision/bsds/BSDS300/html/dataset/images/color/210088.html

Processamento de imagens coloridas

Intensity processing

- Equalização do histograma

```
% color_09 [script]
clear, clc, close all
% Equalização do histograma
% Converter de RGB para um espaço de cores que separe a informação
% de luminância da informação de cor (cromaticidade),
% realizar a operação apenas no canal de luminância e
% converter para RGB.
% Imagem: https://www2.eecs.berkeley.edu/Research/
% Projects/CS/vision/bsds/BSDS300/html/dataset/
% images/color/210088.html
rgb = imread('210088.jpg');
figure, imshow(rgb), title('rgb')

hsv = rgb2hsv(rgb);
v = hsv(:,:,3);
figure, imshow(v), title('v')
v_Eq = histeq(v,256);
figure, imshow(v_Eq), title('v\_Eq')

hsv_Eq = cat(3,hsv(:,:,1),hsv(:,:,2),v_Eq);
rgb_Eq = hsv2rgb(hsv_Eq);
figure, imshow(rgb_Eq), title('rgb\_Eq')
```

rgb

Imagem: Berkeley BSDS300 dataset https://www2.eecs.berkeley.edu/Research/Projects/CS/vision/bsds/BSDS300/html/dataset/images/color/210088.html

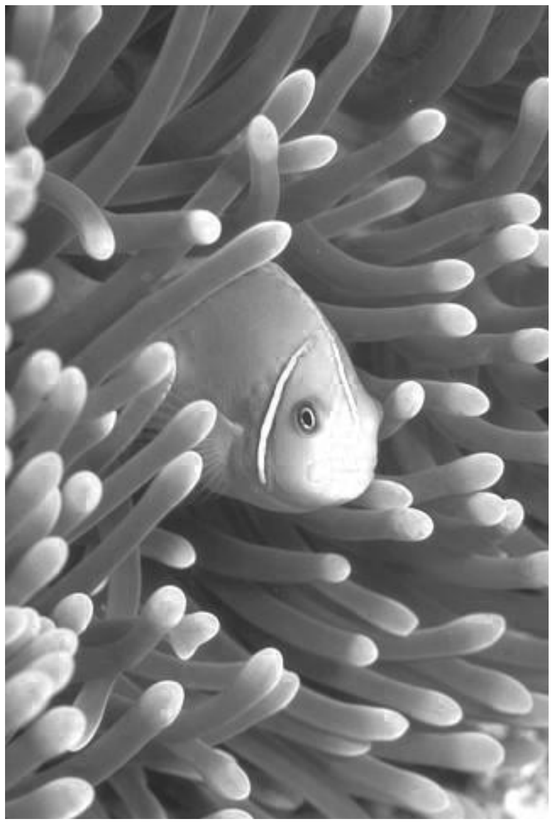
Processamento de imagens coloridas

Intensity processing

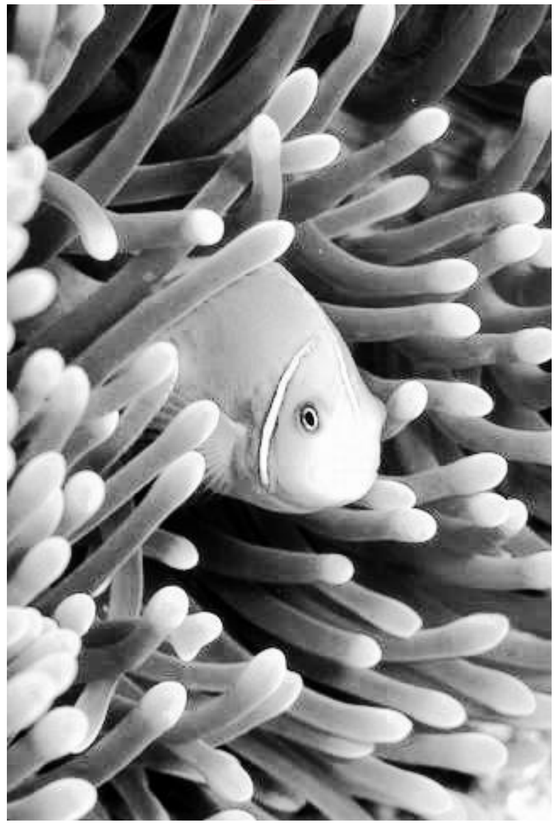
- Equalização do histograma

rgb

v



v_Eq



rgb_Eq

Imagem: Berkeley BSDS300 dataset https://www2.eecs.berkeley.edu/Research/Projects/CS/vision/bsds/BSDS300/html/dataset/images/color/210088.html

Processamento de imagens coloridas

Intensity processing

- Equalização do histograma

rgb

rgb_E

Resultado usando Y do YIQ

rgb_E

Resultado usando H do HSV

Imagem: Berkeley BSDS300 dataset https://www2.eecs.berkeley.edu/Research/Projects/CS/vision/bsds/BSDS300/html/dataset/images/color/210088.html

Processamento de imagens coloridas

21:22
21:37 15 min

- Detecção de bordas

```
% color_10 [script]
clear, clc, close all
% Detecção de bordas:
% Método 1) Converter para grayscale e aplicar a detecção de bordas.
% Método 2) Aplicar o detector de bordas em R, G, B, individualmente
%           e depois combinar os três com uma operação OU lógica.
% Imagem: https://www2.eecs.berkeley.edu/Research/Projects/CS/vision/
%         bsds/BSDS300/html/dataset/images/color/302003.html
rgb = imread('302003.jpg');
figure, imshow(rgb)
r = rgb(:,:,1); g = rgb(:,:,2); b = rgb(:,:,3);
r_g_b = [r g b];
figure, imshow(r_g_b), title('r\_g\_b')

% Método 1)
gray = rgb2gray(rgb);
figure, imshow(gray), title('gray')
gray_c = edge(gray, 'canny');
figure, imshow(gray_c), title('Canny em gray')

% Método 2)
r_c = edge(r, 'canny');
g_c = edge(g, 'canny');
b_c = edge(b, 'canny');
r_g_b_c = [r_c g_c b_c];
figure, imshow(r_g_b_c), title('Canny em R,G,B')
rgb_e = r_c | g_c | b_c;
figure, imshow(rgb_e), title('Canny em R,G,B -> OR')
```

rgb

Imagem: Berkeley BSDS300 dataset https://www2.eecs.berkeley.edu/Research/Projects/CS/vision/bsds/BSDS300/html/dataset/images/color/302003.html

Processamento de imagens coloridas

- Detecção de bordas

rgb

R r_g_b G B

gray

Canny em R,G,B

R G B

Canny em gray

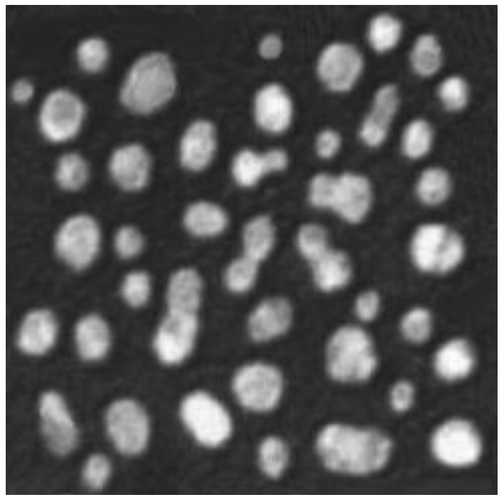
Canny em R,G,B → OR

Imagem: Berkeley BSDS300 dataset https://www2.eecs.berkeley.edu/Research/Projects/CS/vision/bsds/BSDS300/html/dataset/images/color/302003.html

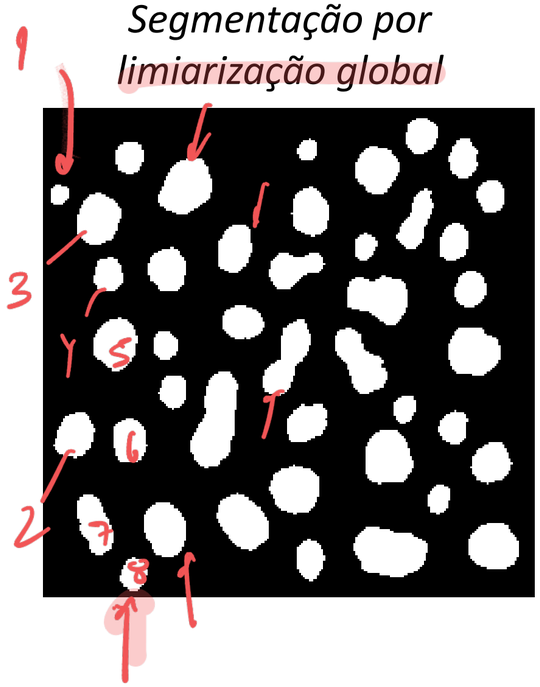
Pseudocolorização (pseudocoloring)

- Aprimorar a visualização de imagens níveis de cinza

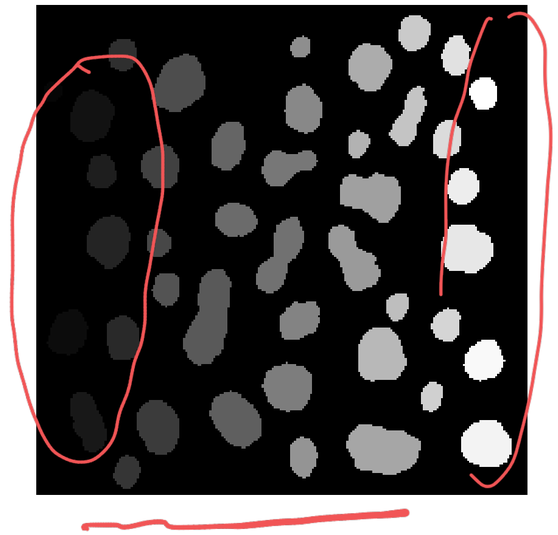
Entrada



Segmentação por limiarização global



Rotulação (labeling)



Pseudocolorização
MATLAB: `label2rgb()`

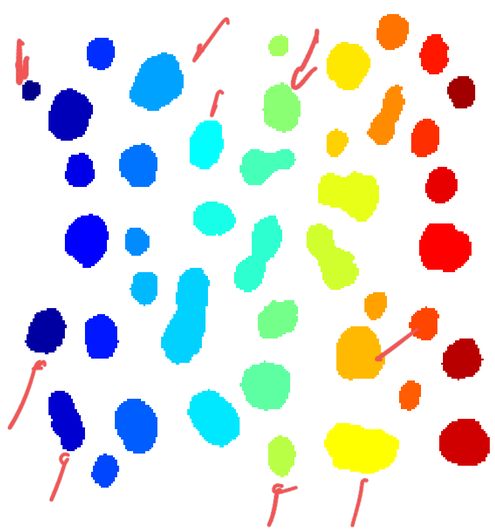
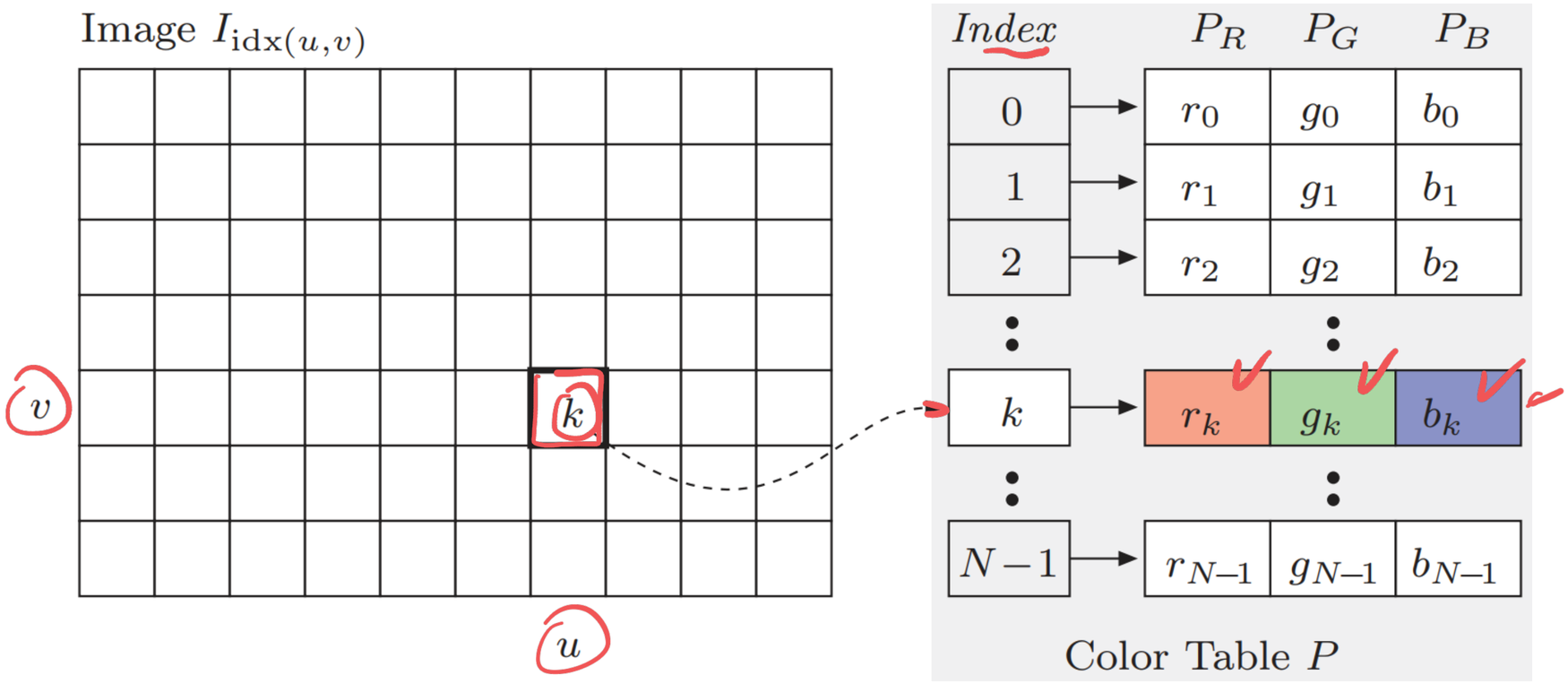


Imagem adaptada de: Geoff Dougherty, Digital Image Processing for Medical Applications, Cambridge University Press, 2009, Figure 9.11.
https://www.cambridge.org/br/academic/subjects/engineering/biomedicalengineering/digital-image-processing-medical-applications?format=HB&isbn=9780521860857

Pseudocolorização (pseudocoloring)

- Imagem indexada (indexed image)



A pseudocolorização não recupera informações de cor presentes na cena original.

Ela estabelece uma correspondência: $g(x, y) \rightarrow [R(g), G(g), B(g)]$

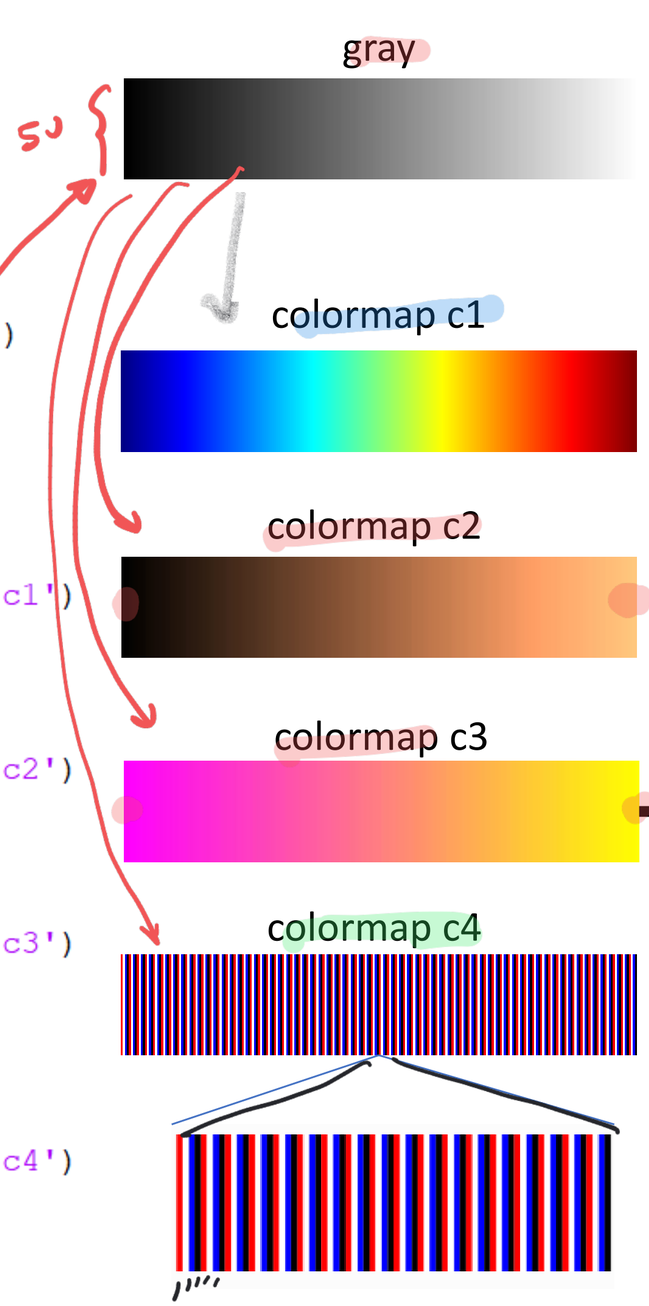
A pseudocolorização é uma operação de mapeamento de intensidades para cores, e não um procedimento de recuperação da cor original.

Figura: Wilhelm Burger, Mark Burge, Digital image processing – an algorithmic introduction using Java, Springer, 2010, Fig 12.5.

Pseudocolorização (pseudocoloring)

- Colormaps

```
% color_11 [script]
clear, clc, close all
% Pseudolorização e Indexed image
% Todos os níveis de cinza em
% uma imagem 50-by-256
gray = uint8(0:255);
gray = repmat(gray, 50, 1);
figure, imshow(gray), title('gray')
% https://www.mathworks.com/help/
% matlab/ref/colormap.html
% traversing B, C, G, Y, R
figure
c1 = colormap(jet(256));
imshow(gray, c1), title('colormap c1')
% traversing K to light copper
figure
c2 = colormap(copper(256));
imshow(gray, c2), title('colormap c2')
% traversing magenta to yellow
figure
c3 = colormap(spring(256));
imshow(gray, c3), title('colormap c3')
% cyclic 4-color map of
% red, white, blue, black
figure
c4 = colormap(flag(256));
imshow(gray, c4), title('colormap c4')
%Para salavar a imagem:
imwrite(gray, c1, 'gray_jet.png');
```



c3

	R	G	B
1	1	0	1
2	1	0.0039216	0.99608
3	1	0.0078431	0.99216
4	1	0.011765	0.98824
5	1	0.015686	0.98431
6	1	0.019608	0.98039
7	1	0.023529	0.97647
8	1	0.027451	0.97255
⋮			
249	1	0.97255	0.027451
250	1	0.97647	0.023529
251	1	0.98039	0.019608
252	1	0.98431	0.015686
253	1	0.98824	0.011765
254	1	0.99216	0.0078431
255	1	0.99608	0.0039216
256	1	1	0

R+B→magenta

B vai diminuindo
G vai aumentando

R+G→yellow

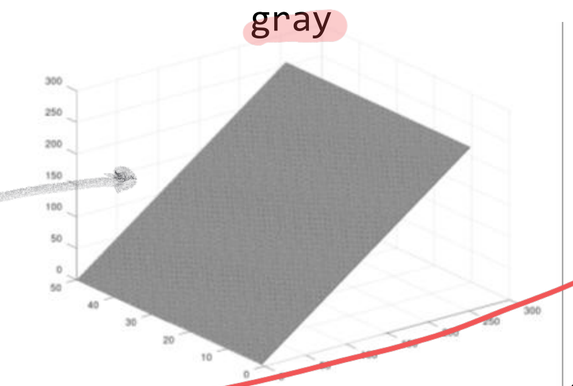
Para obter uma imagem RGB:

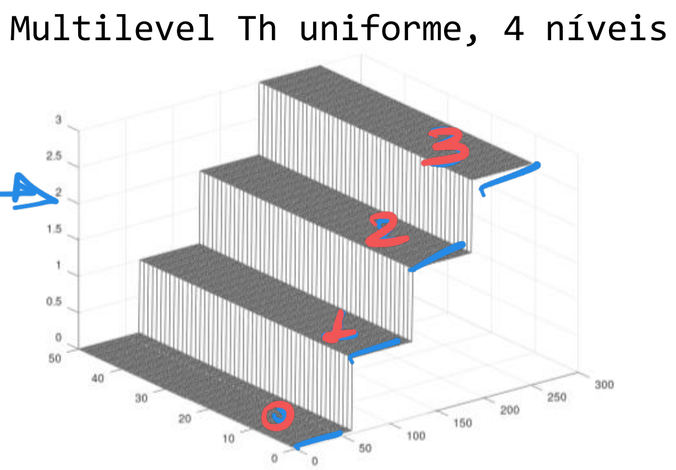
```
RGB = ind2rgb(gray, c1);
imwrite(RGB, 'gray_jet_RGB.png');
```

Pseudocolorização (pseudocoloring)

Colormap discretizado: diferença entre utilizar uma colormap contínua de 256 cores e uma colormap discreta de apenas quatro cores.

```
% Pseudocolor_Quantizado.m
clc; clear; close all;
%Todos os níveis de cinza em uma
%imagem 50-by-256
gray = uint8(0:255);
gray = repmat(gray, 50, 1);
figure;
mesh(double(gray), 'EdgeColor', 'black');
title('gray');
figure;
imshow(gray);
title('gray');
% spring: transição de magenta para amarelo
figure;
c1 = colormap(spring(256));
imshow(gray, c1);
title('Colormap spring 256 cores');
% Multilevel thresholding
% [0,63] -> 0
% [64,127] -> 1
% [128,191] -> 2
% [192,255] -> 3
gray4mth = grayslice(gray, 4);
figure;
mesh(double(gray4mth), 'EdgeColor', 'black');
title('Multilevel Th uniforme, 4 niveis');
figure;
c2 = colormap(spring(4));
imshow(gray4mth, c2);
title('Colormap spring 4 cores');
```

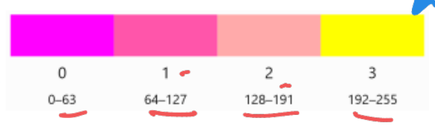




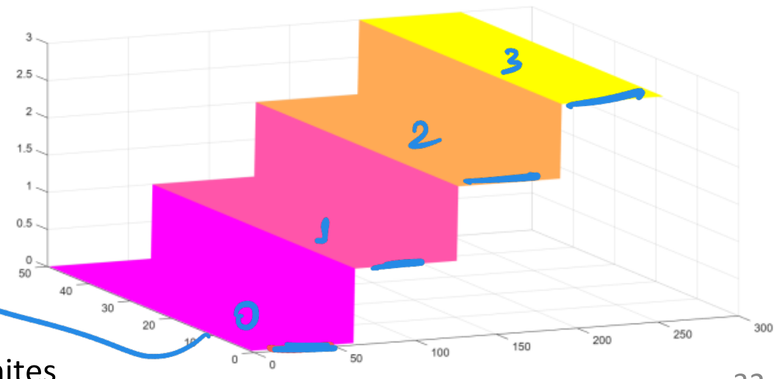
c2

	1	2	3
1	1	0	1
2	1	0.33333	0.66667
3	1	0.66667	0.33333
4	1	1	0

```
figure
surf(double(gray4mth), ...
'EdgeColor', 'none', ...
'FaceColor', 'flat');
colormap(spring(4));
clim([0 3]);
```



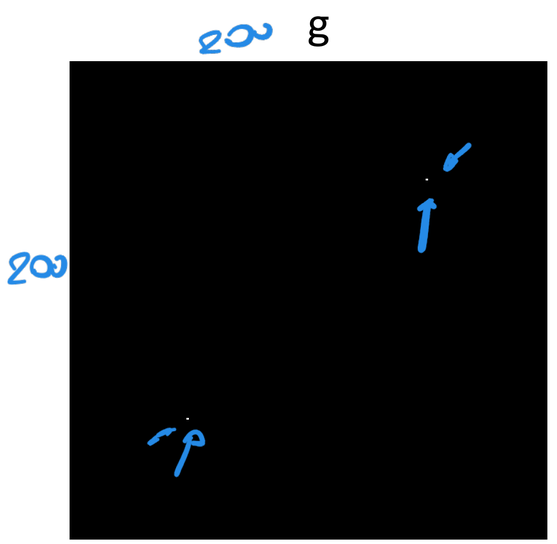
clim() c=Color
lim = limites

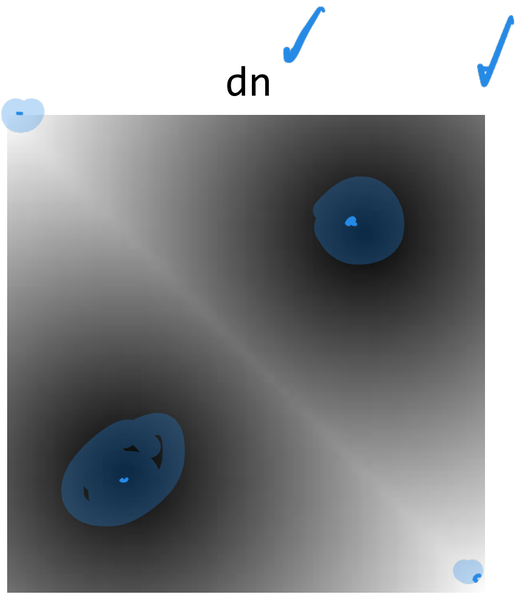


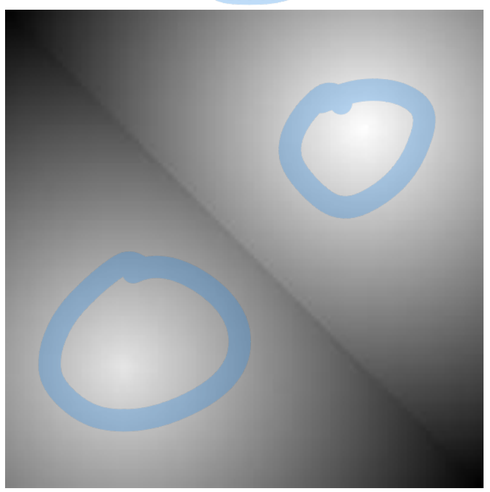
Pseudocolorização (pseudocoloring)

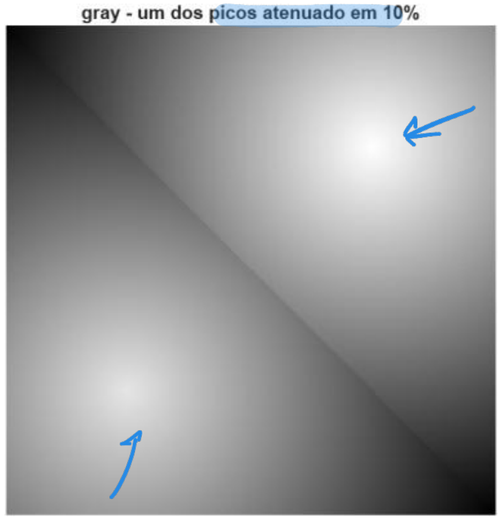
- Preparação da imagem para demonstrar o método intensity slicing

```
%Intensity_Slicing.m
clc; clear; close all;
% imagem com dois pixels de intensidade 1
g = zeros(200, 200);
g(50,150) = 1;
g(150,50) = 1;
figure;
imshow(g);
title('g');
% Transformada da distancia: Para cada
% pixel, calcula a distancia euclidiana
% ate o pixel de valor 1 mais proximo
d = bwdist(g);
dn = mat2gray(d);
figure;
imshow(dn);
title('dn');
% Os dois pontos passam a representar picos
dnn = 1 - dn;
figure;
imshow(dnn);
title('dnn');
%Atenua 10% uma das elevações
t = tril(ones(200,200))*0.9;
t(~t) = 1;
dnn = dnn.*t;
gray = uint8(dnn.*255);
figure;
imshow(gray);
title('gray - um dos picos atenuado em 10%');
```





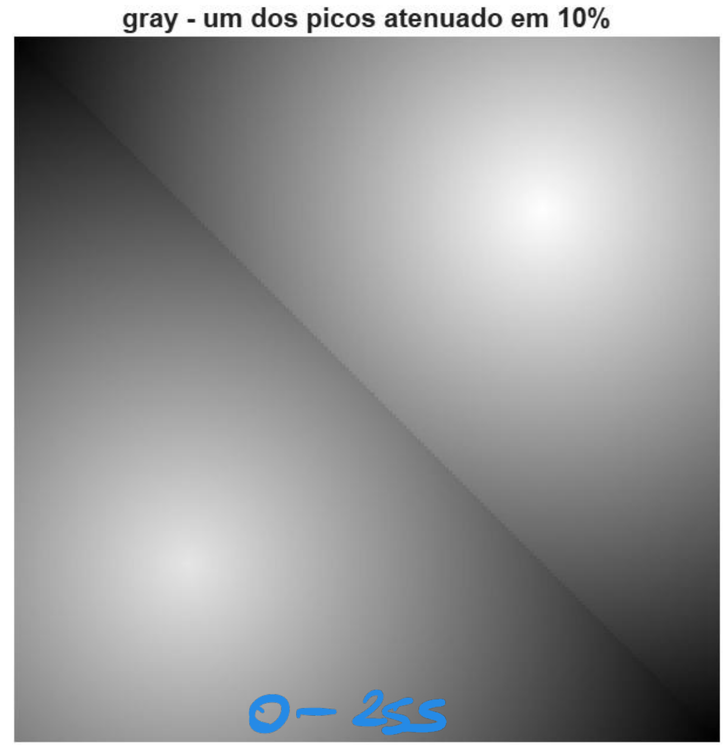


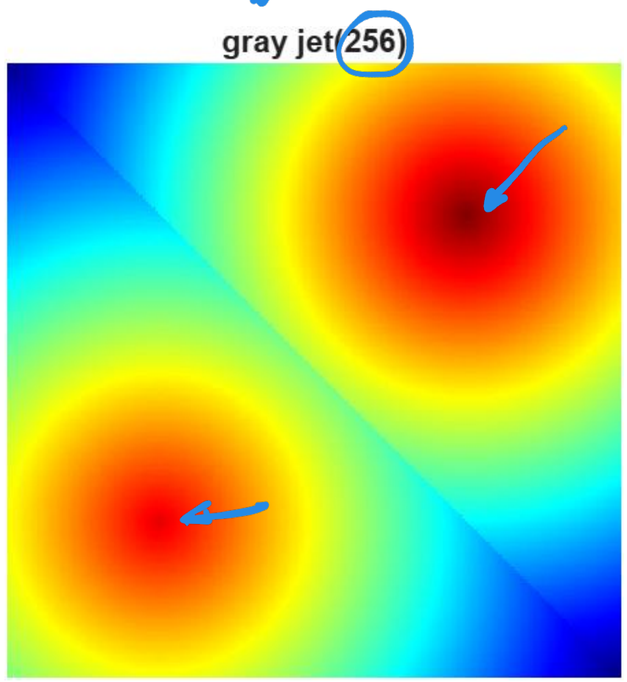


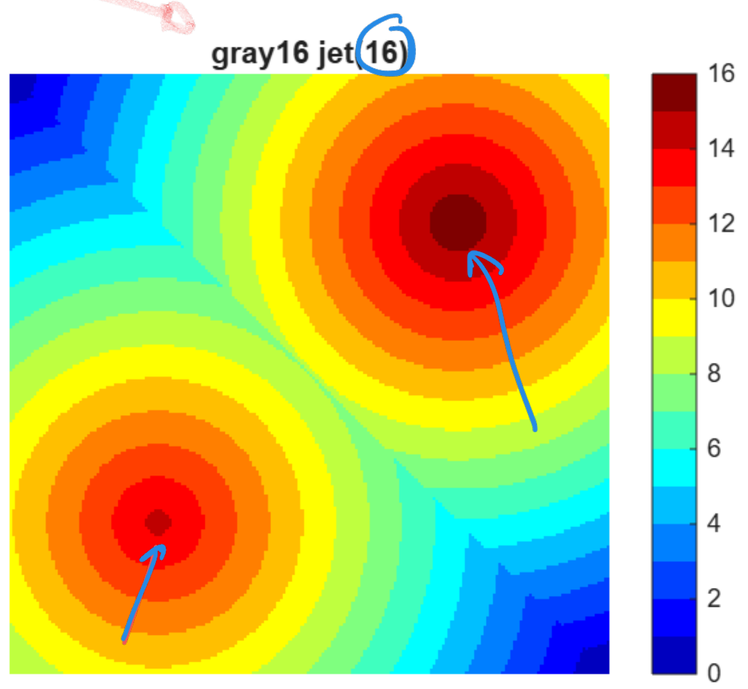
Pseudocolorização (pseudocoloring)

- Método *intensity slicing* para pseudocolorização

```
%Pseudocolorização com 256 cores
figure;
c1 = jet(256);
imshow(gray, c1);
title('gray jet(256)');
%Quantização em 16 níveis
gray16 = grayslice(gray, 16);
%Pseudocolorização com 16 cores
figure;
c2 = jet(16);
imshow(gray16, c2);
title('gray16 jet(16)');
colorbar;
```





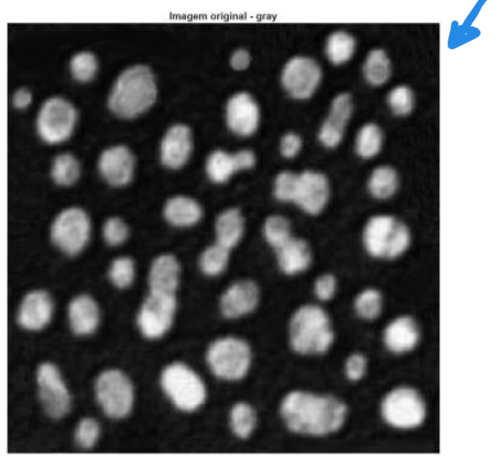


Pseudocolorização (pseudocoloring)

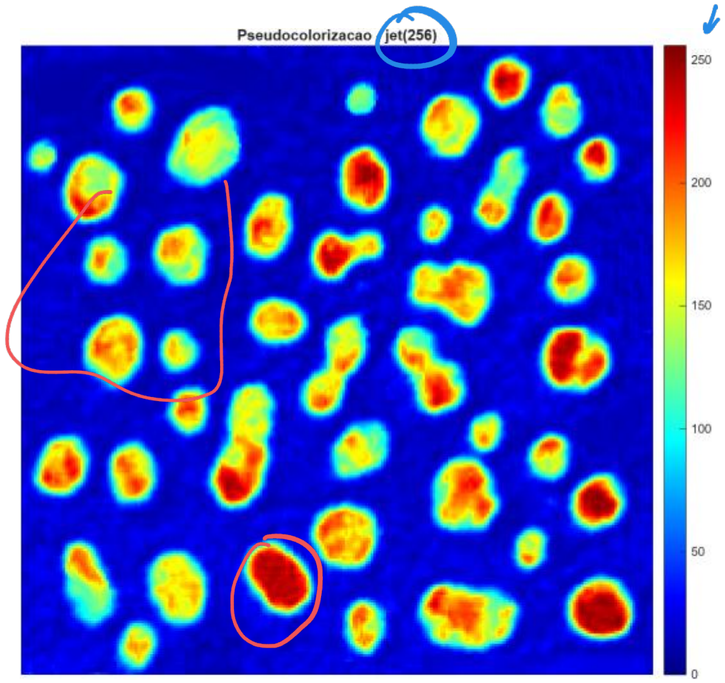
Intensity slicing: uma forma de pseudocolorização por faixas de intensidade, com limiares não uniformemente distribuídos.

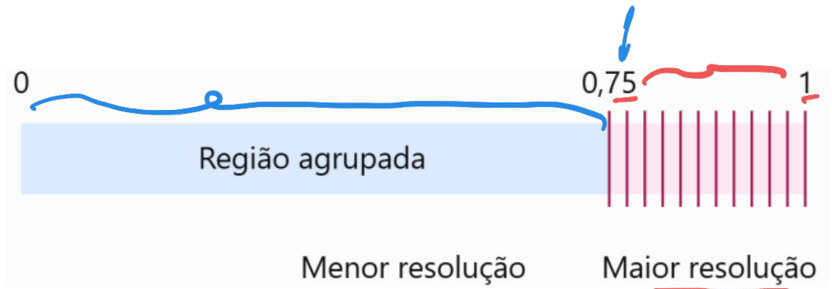
```
% Pseudocolor_Nao_Uniforme.m
% Quantizacao nao uniforme
clc; clear; close all;
gray = imread('whitecells4.png');
grayd = mat2gray(gray);
gray = uint8(grayd.*255);
figure;
imshow(gray); title('Imagem original - gray');

figure;
c1 = jet(256);
imshow(gray, c1);
title('Pseudocolorizacao - jet(256)');
colorbar;
% Parametros para quantizacao nao uniforme
limite_inferior = 0.75; % Inicio da regiao de interesse
limite_superior = 1.00; % Fim da regiao de interesse
N = 12; % Numero de classes acima do limite inferior
slices = limite_inferior + ...
(0:N-1)*(limite_superior-limite_inferior)/N;
% Exibir os limiares na janela de comandos
disp('Limiares utilizados:');
disp(slices');
gray13 = grayslice(grayd, slices);
figure;
c2 = jet(N+1);
imshow(gray13, c2);
title(sprintf('Intensity slicing - %d classes', N+1));
colorbar;
```



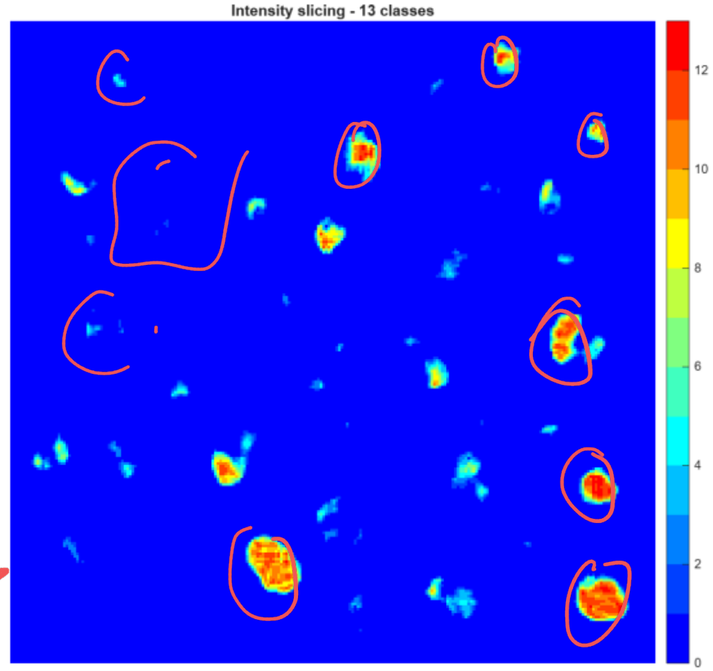
256





A faixa de 0 a 0,75 é agrupada em uma única classe; a faixa de 0,75 a 1 é subdividida em 12 classes.

22h 10min



As regiões com intensidade inferior a 0,75 recebem uma única cor. As com intensidade igual ou superior a 0,75 são diferenciadas em 12 faixas de cores.